BLOCKCHAIN Experiment No: 6

Harshvardhan Patil
D20A/22

- **Aim**: Creation and Deployment of Smart Contract and Transactions using Solidity and Remix IDE.
- **Course Outcomes (CO): CO3, CO5, CO6**
- **Theory**:

Theory

1. Relevance of require Statements in Solidity Programs

In the Property Registration System, the `require` statement acts as a guard condition inside smart contract functions. It ensures that only valid inputs and authorized users can perform property-related operations. If the condition inside `require` is not satisfied, the function execution stops immediately and the transaction is reverted.

In the Property Registration System, `require` can be used to check:

- Whether only the registrar is allowed to register a new property.
- Whether the property owner address is valid before registering a property.
- Whether the property area is greater than zero.
- Whether the property ID exists before retrieving or transferring property details.
- Whether only the current owner is allowed to transfer the property.
- Whether the new owner address is valid before transferring ownership.

For example:

require(msg.sender == registrar, "Only registrar can register property.");

This ensures that only the authorized registrar can register a property.

Another example is:

require(msg.sender == property.owner, "Only current owner can transfer property.");

This ensures that only the current ownerof a property can transfer its ownership.

Thus, `require` statements provide security, correctness, and reliability to the Property Registration System by preventing unauthorized and invalid transactions. They also provide error messages that make contract interaction easier to understand.

2. Keywords: mapping, storage, and memory

mapping

A `mapping` is a special data structure in Solidity that links a key to a value, similar to a hash table. It is useful for storing and retrieving property and owner information efficiently.

In the Property Registration System, the following mapping is used:

mapping(uint256 => Property) public properties;

Here, each Property ID is mapped to a `Property` structure containing information such as property name, location, area, owner, and registration status.

Another mapping used is:

mapping(address => uint256[]) public ownerProperties;

Here, each owner's blockchain address is mapped to an array containing the Property IDs owned by that address.

Mappings are useful because property information can be accessed directly using its Property ID or owner's address. They are efficient for lookups but do not have a length property and cannot be iterated over directly.

storage

In Solidity, `storage` refers to the permanent storage of the smart contract on the blockchain. Data stored in storage remains available across transactions unless it is modified.

In the Property Registration System, property information is permanently stored in the `properties` mapping.

For example:

Property storage property = properties[_propertyId];

Here, `property` is a storage reference to the actual property record on the blockchain. Any modification made through this reference directly changes the stored property information.

For example:

property.owner = _newOwner;

updates the ownership information permanently on the blockchain.

Since blockchain storage is persistent and writing data to storage consumes gas, it should be used carefully.

memory

`memory` is a temporary data location used only during the execution of a function. Data stored in memory is discarded after the function execution ends.

In the Property Registration System, `memory` is used for temporary string inputs such as the property name and location.

For example:

```
function registerProperty(
    string memory _propertyName,
    string memory _location,
    uint256 _area,
    address _owner
) public
```

Here, `_propertyName` and `_location` are stored temporarily in memory while the function executes.

It is also used when retrieving property information:

Property memory property = properties[_propertyId];

The property data is copied into memory for temporary use.

Thus, `storage` is used for permanent blockchain data, while `memory` is used for temporary data during function execution.

3. Why string Instead of bytes32?

In the original Ballot contract, `bytes32` was used to store short proposal names. For the Property Registration System, `string` is more suitable because property names and locations can have different lengths and are easier for users to understand.

For example:

string propertyName;
string location;

A property may have a name such as:

Green Valley Apartment

and a location such as:

Mumbai, Maharashtra

Using `string` allows the Property Registration System to store text of variable length without restricting the user to a fixed 32-byte value.

`bytes32` is a fixed-size data type that can store exactly 32 bytes. It can be useful when working with short, fixed-length values and can be more efficient in some situations. However, it is less convenient for property names and locations because the values must fit within the fixed size.

`string`, on the other hand, is a dynamically sized data type and is more suitable for human-readable information such as:

- Property Name
- Property Location
- Owner-related text information

Therefore, the Property Registration System uses `string` instead of `bytes32` to make property information user-friendly, readable, and flexible.

In this project, `string` is preferred for property-related text, while Solidity's other appropriate data types such as `uint256`, `address`, and `bool` are used for property ID, area, ownership address, and registration status respectively.

Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract PropertyRegistration {
    struct Property {
        uint256 propertyId;
        string propertyName;
        string location;
        uint256 area;
        address owner;
        bool registered;
    }
```

```
    mapping(uint256 => Property) public
properties;

    mapping(address => uint256[]) public
ownerProperties;

    uint256 public propertyCount;

    address public registrar;

    constructor() {
        registrar = msg.sender;
    }

    function registerProperty(
        string memory _propertyName,
        string memory _location,
        uint256 _area,
        address _owner
    ) public {
        require(msg.sender == registrar, "Only
registrar can register property.");
        require(_owner != address(0), "Invalid
owner address.");
        require(_area > 0, "Area must be greater
than zero.");

        propertyCount++;

        properties[propertyCount] = Property(
            propertyCount,
            _propertyName,
            _location,
            _area,
            _owner,
            true
        );


ownerProperties[_owner].push(propertyCount);
    }

    function transferProperty(
        uint256 _propertyId,
        address _newOwner
    ) public {
        require(_propertyId > 0 && _propertyId <=
propertyCount, "Invalid property ID.");
        require(_newOwner != address(0), "Invalid
new owner address.");

        Property storage property =
properties[_propertyId];

        require(
            msg.sender == property.owner,
            "Only current owner can transfer
property."
        );

        property.owner = _newOwner;


ownerProperties[_newOwner].push(_propertyId)
;
    }

    function getProperty(
        uint256 _propertyId
    )
        public
        view
        returns (
            uint256,
            string memory,
            string memory,
            uint256,
            address,
            bool
        )
    {
        require(
            _propertyId > 0 && _propertyId <=
propertyCount,
            "Property does not exist."
        );
```

OUTPUT

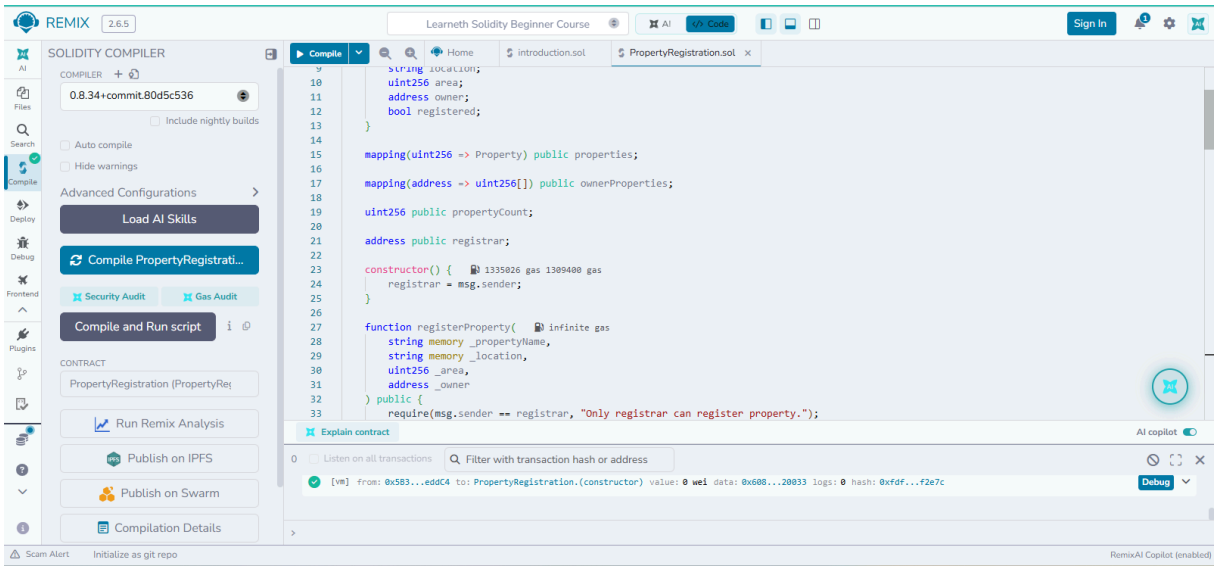


Figure 1: Successful compilation of PropertyRegistration.sol in Remix IDE.

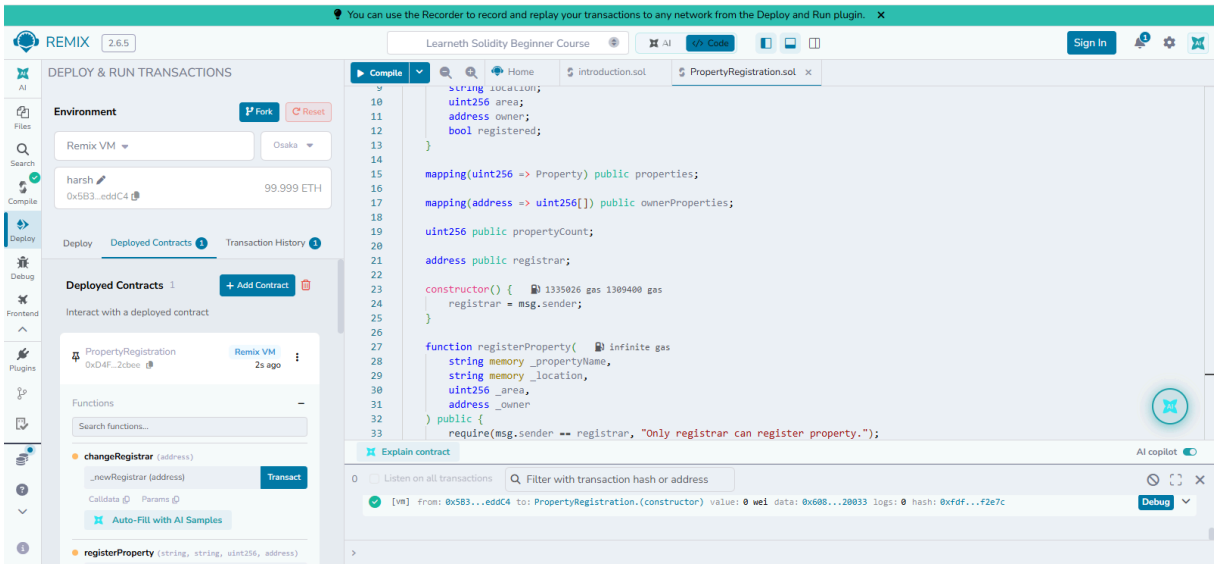


Figure 2: Deployment of the PropertyRegistration smart contract in Remix VM.

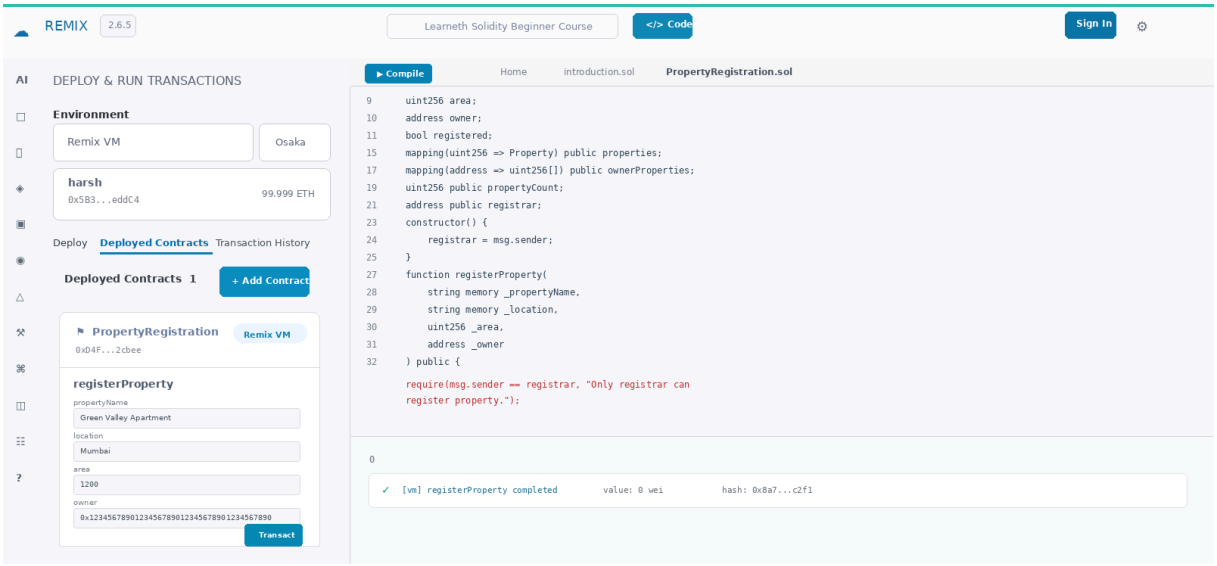


Figure 3: Registering Green Valley Apartment using the registerProperty transaction.

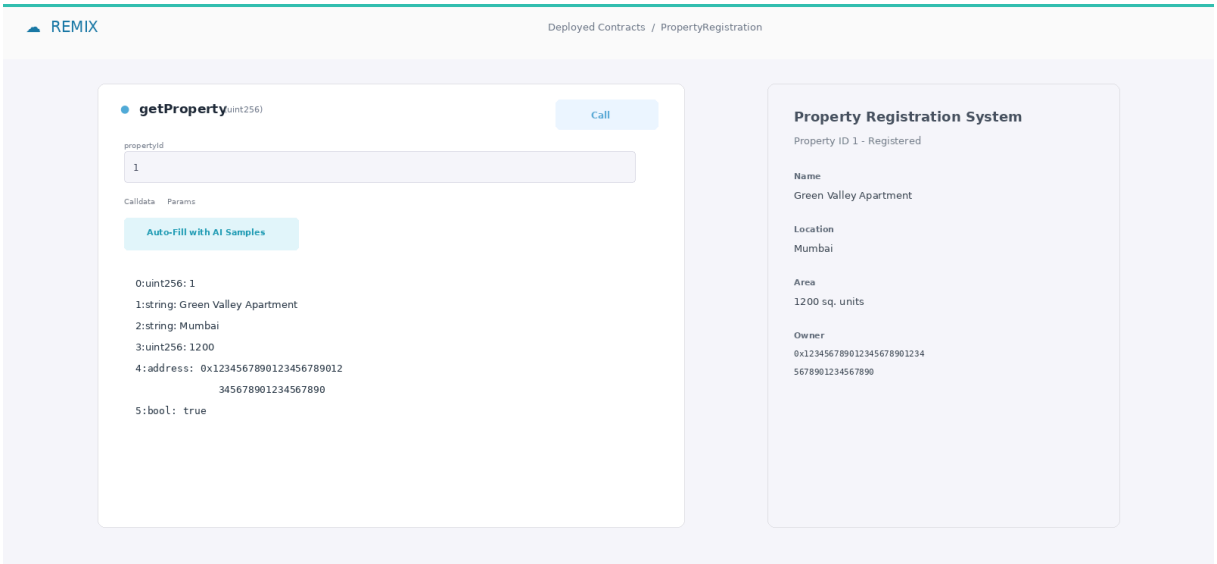


Figure 4: Viewing the registered property details using getProperty(1).

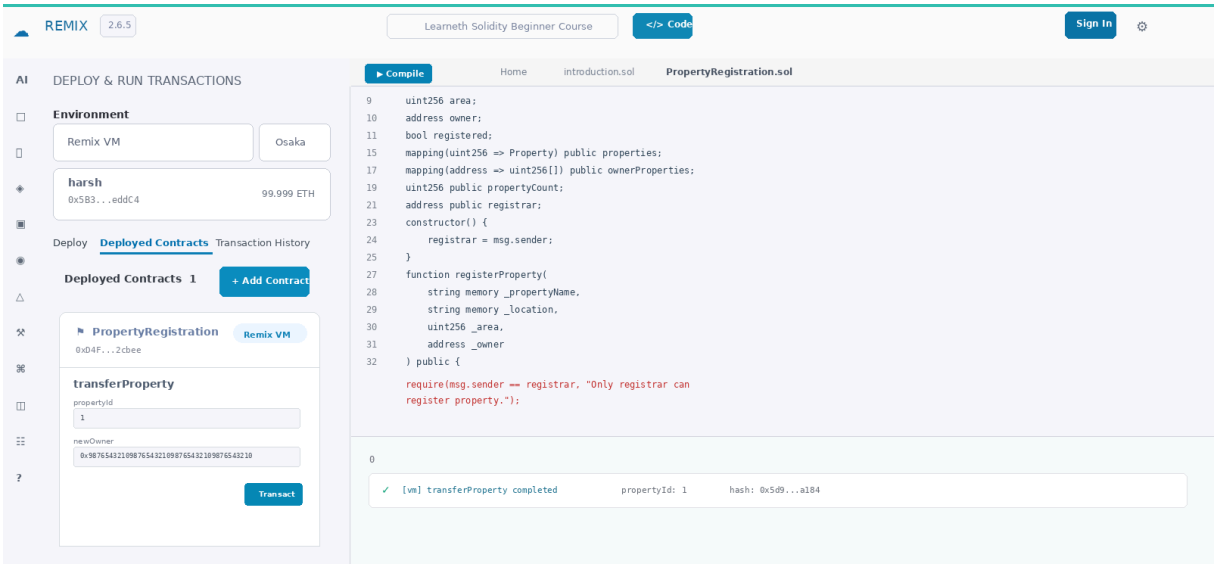


Figure 5: Transferring Property ID 1 to a new owner using transferProperty.

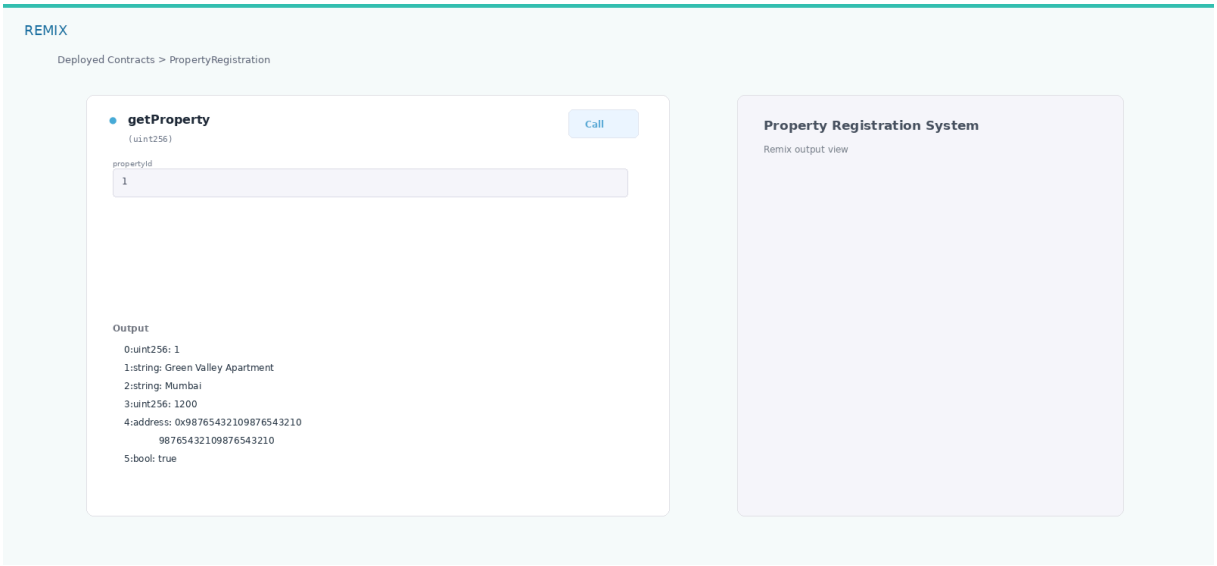


Figure 6: Updated property details showing the new owner after transfer.

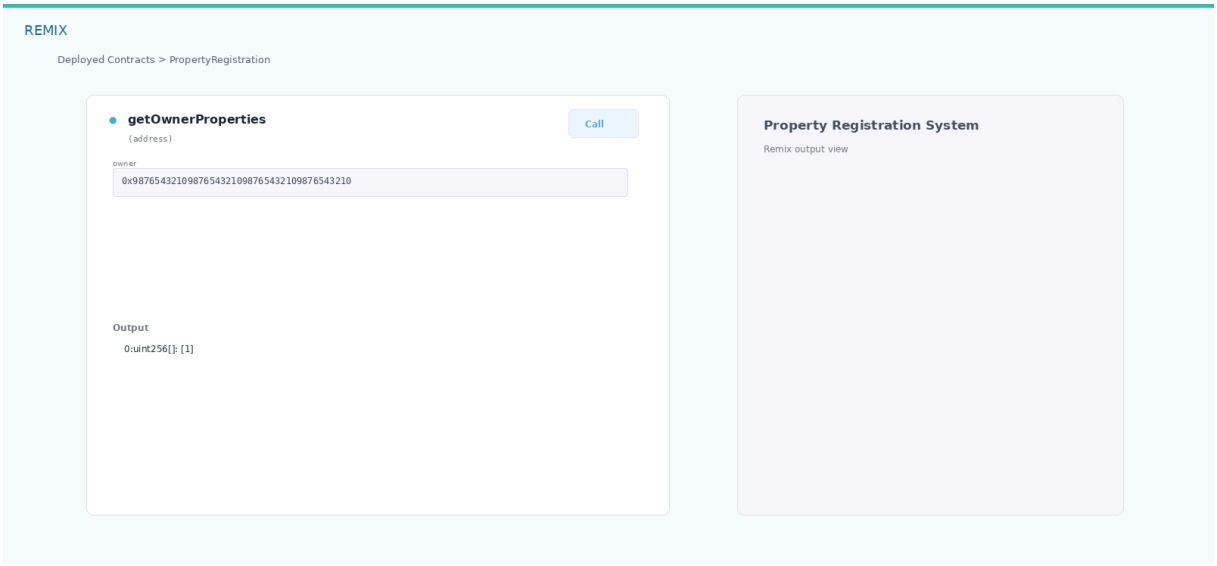


Figure 7: Retrieving the property IDs mapped to the new owner.

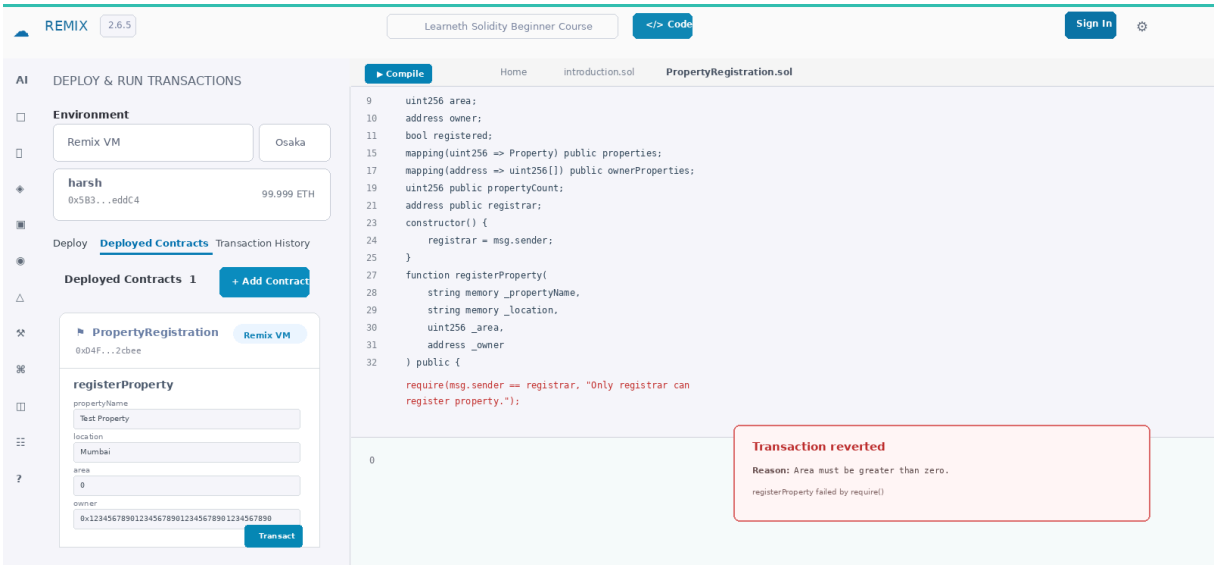


Figure 8: require() validation rejecting an invalid property registration transaction.

Conclusion:

In this experiment, the Property Registration smart contract was successfully created, compiled, deployed, and tested using Solidity and Remix IDE. Property registration, property-detail retrieval, ownership transfer, and owner-property mapping were demonstrated through blockchain transactions. The require statements were also tested to validate inputs and restrict unauthorized operations. The experiment provided practical understanding of smart-contract deployment and transaction handling while applying mapping, storage, and memory concepts to a Property Registration System.